

UNICONES User's Manual

1. Introductions

The Universal Conservation Equation Solves (UNICONES) is a computational framework for large-scale simulation of general physical conservation laws based on the space-time conservation element, solution element method. The code suite along with pre- and post-processing tools provide high-fidelity time-accurate flow simulations and allows users to plug in her own physical conservation laws for space-time integration using the software framework of UNICONES. Noted, in the following Courier fonts are used to indicate input/output texts and filenames.

There are several ways to run UNICONES.

The first one is to run the code in serial mode interactively, which can be done by entering the following syntax in the command line or executing a script file containing the following line:

```
unicones -t casetag case.in >& case.out
```

where a script called `case.in` is required. The tag name associated with this run can be any chosen tag specified in `casetag`. For example, a run like:

```
unicones -t .run88 run88.in >& run88.out
```

will use a prepared script called `run88.in` to execute the code. The preparation of the input script file will be described in the next section. This run will generate a file called `run88.out` which contains all stdout for this execution. The following output files will be generated:

```
unicones.enorm.case88 (convergence norm data, to be used for restart)
twalls_values.dat.case88 (wall values after the run)
xtecp_sol_allz.dat.case88 (Tecplot™ solution file)
binDumpZ.dat.case88 (zone dump files to be used for restart)
```

Running the code in an interactive mode can be terminated by using `<cntl>-C` or accidentally logging out.

The second way is to run the code in serial mode in the background, which can be done by entering the following syntax in the command line or executing a script file containing the following line:

```
nohop time unicones -t casetag case.in >& case.out
```

The run will not continue even user log out the system. The way to stop the job is to type the command “`kill -9 #####`”, where `#####` is the process number of the job, and users can find out the info by using “`ps aux | grep unicones`”.

The third way is to run the code in parallel mode, which can be done by entering the following syntax in the command line or executing a script file containing the following line:

```
mpirun -np $$ unicones -t casetag case.in >& case.out
```

`$$` is the number of processors users want to use to compute the case in parallel computing.

The fourth way is to run the code in parallel mode in the background, which can be done by entering the following syntax in the command line or executing a script file containing the following line:

```
nohop time mpirun -np $$unicones -t casetag case.in >&  
case.out
```

Lastly, the aforementioned different ways of running the code can also be done by submitting a run job using `sbatch` (in Slurm system) or `qsub` (in PBS system) with a run script file. Detailed descriptions about the batch file format, submitting/monitoring/deleting the run job and other options, users should refer to Slurm or PBS system manual.

2. Input Script Preparation and Descriptions

2.1 Overview of input scripts:

The input script file defines each run by a (parameter, value) pairs prepared by the user.

Allowable parameter types:

1. ENUM (C++ type), equivalent to integer
2. BOOLEAN: true or false
3. STRING_ARRAY: self-explanatory
4. NUMBER: a real or integer number
5. NUMBER_ARRAY: a number array

All valid input parameters and their associated types are grouped into 6 categories and are listed below. Detailed descriptions of the input scripts and their associated parameters are provided in Section 2.2.

Category 1: General problem definition and MPI related scripts

Input Parameter	Parameter type	Default
DomainType	ENUM	E2D
mpi_use_lean_mem	BOOLEAN	false
mpi_dump_each_rank	BOOLEAN	false
mpi_comm_map_fn	STRING	NULL
readin_iface_info	BOOLEAN	false
process_iface_info	BOOLEAN	false
apply_pthread	BOOLEAN	false
max_nthread	NUMBER	1
num_zones_per_block	NUMBER	1
solve_gcl	BOOLEAN	false

Category 2: Instruction scripts for I/O and initialization

Input Parameter	Parameter type	Default
grid_type	ENUM	unstructured
grid_fname	STRING	must provide
mixed_mesh	BOOLEAN	false
bc_fname	STRING_ARRAY	must provide
restart	BOOLEAN	false
convg_fn_restart	STRING	"unicones.enorm"
zdump_fn_restart	STRING	"binDumpZ.dat"
init_cond_use_zdump_fn	STRING	empty string
init_cond_use_zdump_fn_usedll	BOOLEAN	false
probe_points_def_fn	STRING	empty string
probe_output_skip_niter	NUMBER	1
num_iter	NUMBER	100

print_sol_per_niter	NUMBER	1
print_convrg_per_niter	NUMBER	1
dump_zone_per_niter	NUMBER	1
print_sol_phys_time	NUMBER	No default value
dump_zones_phys_time	NUMBER	No default value
tecplot_write_vtx_output	BOOLEAN	false
tecp_out_each_zone	BOOLEAN	false
calc_time_mean_values	BOOLEAN	false
mean_flow_zdump_fn	STRING	empty string
write_wall_force	BOOLEAN	false
process_one_zone_aat	BOOLEAN	false

Category 3: Instruction scripts for flow conditions/variables

Input Parameter	Parameter type	Default
flow_solu_type	ENUM	ns_sol
Transp_prop_option	ENUM	CONST_PRANDTL
fs_Mach	NUMBER	0.5
fs_Tinf	NUMBER	100
fs_UnitRe	NUMBER	25400
fs_flow_angle	NUMBER	0
Linf	NUMBER	1
gamma	NUMBER	1.4
Rgas	NUMBER	287
stokes	NUMBER	0
Prandtl	NUMBER	0.7
axisymmetric_2D	BOOLEAN	false
init_cond_depQ	NUMBER_ARRAY	default not used

Category 4: Instruction scripts for controlling numerical computations and dissipations

Input Parameter	Parameter type	Default
use_incenter	BOOLEAN	false
cfl	NUMBER_ARRAY	No default values
constant_cfl	BOOLEAN	false
dt_const	NUMBER_ARRAY	No default values
cfl_schedule_fn	STRING	empty string
unsteady_lstep	BOOLEAN	false
alpha	NUMBER_ARRAY	empty array
epsilon	NUMBER_ARRAY	empty array
beta	NUMBER_ARRAY	empty array
dissip_ctrl	NUMBER_ARRAY	empty array
dissip_per_cfl	BOOLEAN	true
shock_gradp_value	NUMBER	-50
shockf_gradp_beta	NUMBER	-2

shockf_gradp_eps	NUMBER	5
shockf_gradp_dissip_ctrl	NUMBER	1.e-12
Convg_norm_type	ENUM	L2NORM

Category 5: Instruction scripts related to turbulent flow computations

Input Parameter	Parameter type	Default
turb_model_option	ENUM	laminar
construct_wall_dist_db	BOOLEAN	false
read_in_wall_dist_db	BOOLEAN	false
skip_turb_when_dump_zones	BOOLEAN	false
skip_turb_when_read_zones	BOOLEAN	false
output_perturb_solution	BOOLEAN	false
subgrid_turb_fluct_model	BOOLEAN	false
les_wall_model	NUMBER	0
les_vwall_slip_vel	NUMBER	0

Category 6: Instruction scripts for advanced features:

6.A: Immerse boundary method (IBM):

Input Parameter	Parameter type	Default
immersed_boundaries	BOOLEAN	false
SolidPhaseVarIndex	NUMBER	-1 (not used)
SolidPhasePhiMin	NUMBER	0.5
SolidPhaseMotionDLL	STRING	empty string
SolidPhaseAlpha	NUMBER	1.0
SolidPhaseEpsilon	NUMBER	0.
SolidPhaseMuRatio	NUMBER	1.
stationary_solid	BOOLEAN	false
rigid_moving_body	BOOLEAN	false
Weber	NUMBER	0.

6.B: Real gas with multiple chemical species (i.e., variable specific heats):

Input Parameter	Parameter type	Default
To be added		

6.C: Chemical reactions:

Input Parameter	Parameter type	Default
To be added		

6.D: Particle tracking:

Input Parameter	Parameter type	Default

To be added		
-------------	--	--

Noted that as shown above most parameters have a default value, thus only the problem specific inputs need to be specified in the input script. The unspecified parameters in the input script will take the default values. Most frequently used parameters are described below. For a Boolean type parameter, users need to specify true or false.

2.2 Detailed descriptions:

The description of input scripts is detailed below, and some sample input files are provided in Appendix A.

Category 1: General problem definition and MPI related scripts

`DomainType`: E2D (default) or E3D (Euclidean 2 or 3-D)

`mpi_use_lean_mem`: If set true, the “lean” memory option for the block data is turned on. This implies that no post-processing will be done at any MPI rank. All results would be zone dumped for each rank separately for later post-processing. If set false (default), rank 0 has all data from other ranks and post-processing can be done at rank0.

`mpi_dump_each_rank`: If set true, zone dumps for the MPI runs would take place at each rank independently, default false (must set true if `mpi_use_lean_mem` = true).

`mpi_comm_map_fn`: If specified, this file contains the communication maps among all MPI ranks. The map is used to carry out point to point MPI communication of interface zones. If the decompose utility is used to do domain decomposition, this file will be automatically generated.

`readin_iface_info`: This parameter is used only for MPI runs. Interfaces between different grid blocks (assigned to different MPI rank) need to be generated and communications among them need to be set up. Setting this parameter false will cause the code to generate interface information and write it to file for later use. If set true, the generation process is bypassed and interface information will be read in from files (with a default set of file names). This parameter needs to be false for the first MPI run and true for all later runs, assuming the first run generated interface information successfully. Default is false.

`process_iface_info`: In MPI runs, zonal interfaces need to be processed before the run starts so that proper MPI communications can be carried out accordingly. If this parameter is set true, interface zones are matched by the code and the matching data is written to a sequence of files called “Ziface_db.n” where n varies from 0 to N – 1 and N is the total number of MPI blocks (which equals to the total rank). Once these files are generated successfully the code exits. Later MPI runs can then be executed with the option of `readin_iface_info` equals to true. Default is false.

`apply_pthread`: Whether to apply p-thread for in-node parallelization or not (default false)

`max_nthread`: Maximum number of concurrent threads allowed, suggested value is at least greater than number of cores in the node plus 4 (default 1)

num_zones_per_block: Further divide each block (partition) into this number of zones for pthread acceleration (default 1)

solve_gcl: If set true, the general conservation law (GCL) equations compiled in the code (only one set of GCL can be compiled) will be solved instead of the build-in flow solver, default false.

Category 2: Instruction scripts for I/O and initialization

grid_type: Unstructured (triangles/tetrahedrons), or unstructured48 (unstructured quad/hex), default unstructured.

grid_fname: Grid file name

mixed_mesh: Whether to use the mixed mesh format in reading the grid file or not, default is false. Note that the old format is non-mixed mesh. In the mixed_mesh format, the boundary element section has one more input to indicate total number of vertices in a boundary element (see below).

bc_fname: Name of the file containing boundary condition specifications

Restart: Whether a particular run is a restart case or not (default is false). If a restart case, the following two parameters must also be specified in the input script.

convg_fn_restart: Name of the restart convergence file from a previous run

zdump_fn_restart: Name of the zone dump file to be used for restart

init_cond_use_zdump_fn_usedll: If set true, the file specified in init_cond_use_zdump_fn would be interpreted as a dynamically linked library (DLL) file. Corresponding initial conditions everywhere in the domain will be determined by user-supplied codes. If set false, the file specified in init_cond_use_zdump_fn would be interpreted as a zone dump file generated by previous UNICONES runs. Default is false.

init_output_fn: If specified, this file should contain a zone dump file or a DLL file. This zone dump will be used as the initial conditions accordingly, default is an empty string, i.e., not supplied, then initial conditions would be determined by whether init_cond_depQ is present or not.

probe_points_def_fn: If specified, to be used as the file name for probe points definition. This file should contain probe coordinates (x, y) for 2D and (x, y, z) for 3D as one line per point. If not specified, no probe output would be carried out. Default is an empty string, implying not to use probe options.

probe_output_skip_niter: If specified, the probe output would be carried out every so many time iterations. Default is to output at each time iterations.

probe_output_skip_niter:

num_iter: Number of steps to be run (total time = dt_const × num_iter / 2), default 100. The division by 2 in calculating total time is because CESE is a dual-level scheme.

print_sol_per_niter: Print out solutions in Tecplot™ format every so many iterations, print at every iteration if not specified.

print_convg_per_ntier: Print out convergence norms every so many iterations, print at every iteration if not specified.

dump_zone_per_niter: Dump zones for restart every so many half-iterations, dump at every iteration if not specified.

`print_sol_phys_time`: If this number is specified, solutions will be written at the physical time specified, regardless of the iteration count. This is useful for comparison with experiments. Default is unspecified.

`dump_zones_phys_time`: Similar to `print_sol_phys_time`, but applies to zone dump files to be written. Default is unspecified.

`tecplot_write_vtx_output`: If set true, solutions will be written on all cell vertices in the Tecplot output file, otherwise, it will be cell center solutions, default false. For MPI runs, cell center output is more reliable since the CESE method is a cell-centered scheme.

`tecplot_out_each_zone`: If set true, Tecplot™ solutions will be written for all MPI blocks separately. This option does not require combining all blocks into one and thus demands much less memory to post-processing. Default is false.

`calc_time_mean_values`: If set true, time averages will be computed and written to files (“MeanValue_tec.dat” for scalar runs and “MeanValue_dom.*” for MPI runs). Default is false.

`mean_flow_zdump_fn`: If specified, this file will be used to read in mean flowfield and solve for perturbations of all dependent variables. The file format should be exactly like a zone dump file. Suggest using a separate run to achieve steady-state, and impose it as the mean flowfield. Default is unspecified and no mean+perturbation option will be imposed.

`write_wall_force`: If set true, the total viscous wall force will be computed and written to a file called WallForce.dat. Default is false.

`process_one_zone_aat`: If set true, each MPI block is processed independently, saving memory. Default is false.

Category 3: Instruction scripts for flow conditions/variables

`flow_solu_type`: euler_sol, or ns_sol (default ns_sol)

`Transp_prop_option`: Specifies the option for transport property under perfect gas assumptions. Possible choices are `CONST_PROP`, `CONST_PRANDTL`, `VAR_PRANDTL`. Default is `CONST_PRANDTL`.

`fs_Mach`: Freestream Mach number

`fs_Tinf`: Freestream temperature in degree K

`fs_UnitRe`: Freestream unit Reynolds number per meter

`fs_flow_angle`: Freestream flow angle (in 2D) measured by x, y coordinates (default 0)

`Linf`: Length scale used to normalize grid coordinates (in meter), input 1 if dimensional in meters (default 1)

`gamma`: Ratio of specific heats ($\gamma = C_p/C_v$) for perfect gas (default 1.4)

`Rgas`: Perfect gas constant (in SI unit, around 287 in standard conditions), default 287.

`stokes`: Stokes coefficient for the second viscosity (default 0)

`Prandtl`: Prandtl number (default 0.7)

`axisymmetric_2D`: Whether or not to turn on axisymmetric option (default false)

`init_cond_depQ`: Initial conditions to be applied everywhere (in dep. var format, i.e., ρ , ρu , ρv , ρe_t for 2D and ρ , ρu , ρv , ρw , ρe_t for 3D). If not specified, the default initial conditions would be at freestream conditions everywhere in the flowfield, i.e., `init_cond_depQ = 1 1 0` for 2-D ideal-gas flow cases.

Category 4: Instruction scripts for controlling numerical computations and dissipations

`use_incenter`: If set true, use the incenters (instead of geometric centers) of the neighboring elements to form the conservation elements for space-time integration. This choice avoids tips of the CE to be located outside of the domain, which often is for highly curved wall geometries. One major effect of this option is the reduction of numerical dissipation in the edge-based derivatives approach. However, sometimes the results will retain higher amplitudes of high wave number subgrid-scale noise. Default is false.

`cfl`: The desired CFL number to be used for time step determination

`constant_cfl`: To use variable time step at each cell for steady-state calculations, set it to true; to use a uniform time step for each cell for time-accurate computations, set it to false. For `constant_cfl = false`, time step is determined by: (1) if the CFL number is specified in the input file (as `cfl`), use it to calculate time step size at each time level repeatedly, and (2) if no `cfl` is found in the input script, use the parameter `dt_const` specified in the input script. If neither is specified, the code would blow up.

`dt_const`: The constant time step (normalized by L_{inf}/U_{inf}) to be used for time marching, only used if `constant_cfl = false` and `unsteady_lstep = false` (see Note 1 below).

`cfl_schedule_fn`: Specifies the file name for the time step and dissipation control parameter values as a table with respect to the iteration number. Default is an empty string.

Note 0: =====

For certain configurations, initial transient state does not satisfy conservation laws and this makes the initial run very sensitive to time step and dissipation control parameters. The option `cfl_schedule_fn` allows users to set a time step and dissipation parameter schedule that varies with iteration counts. Users can then use smaller time steps in conjunction with stronger numerical dissipation at early stages and gradually ramp up or down the parameters as initial transients passing by the domain. For example the following file “`cfl_schedule.double_cone`” has:

iter	cfl	dt	alpha	beta	dissip_ctrl	epsilon
0	0.01	0	1.	-2000	1.e-12	50.
5000	0.02	0	1.	-2000	1.e-12	30.
10000	0.05	0	1.	-2000	1.e-12	10.
20000	0.10	0	1.	-2000	1.e-12	5.
30000	0.30	0	1.	-2000	1.e-12	3.

Users can then set `cfl_schedule_fn = “cfl_schedule.double_cone”`. Then proper CFL numbers and various dissipation control parameters will be set according to the iteration number from the table. This option can be used both in sequential or MPI runs.

=====

`unsteady_lstep`: Whether to use time-accurate local time stepping (TALTS) or not. If set true, time step at each cell will be automatically computed by `cfl` given in the input.

`alpha`: This parameter ranges from 0 – 2. Larger values of `alpha` imply larger numerical dissipation and is useful to suppress numerical wiggles that generally appear near discontinuities. For acoustic wave simulations and low-speed flow, a very small `alpha` (1E-

12) can be used, amounting to a simple averaging procedure (no need for any wiggle-suppression). For shock-capturing, setting $\alpha = 1$ or 2 is recommended.

beta: positive beta determines the dissipation levels in the freestream region away from a boundary layer or shock regions. For time-accurate runs, beta can be between -1.01 to -1.5 (smaller absolute value for lesser dissipation). Setting beta to a large negative value (say, -5000), will invoke a different way of dissipation control designed for high-aspect ratio boundary-layer meshes typically used for RANS computations (see Note 1 below).

epsilon: indicate the bound of the sigma value outside of the boundary layer edge if beta is positive. If beta is negative and `dissip_ctrl` nearly zero (see Note 1 below), epsilon represent the constant sigma value applies to everywhere in the entire domain.

dissip_ctrl: additional control parameters, see below.

dissip_ctrl_per_cfl: if set true, numerical dissipation control is based on local CFL number, otherwise, it would be based on cell sizes entirely, default true.

Note 1: =====

Numerical dissipation is controlled by adjusting how derivatives are computed using the edge-based derivative approach. Details of numerical dissipation control can be found in AIAA Paper 2007-5818 and AIAA Paper 2013-3069. In this approach, spatial derivatives of the dependent variables are computed by taking averages over the triangles (2D) or tetrahedra (3D) formed by the solution point (the location within the element where the Taylor-series approximation of dependent variables takes the origin) and each neighboring edge (line for 2D, triangles for 3D tetrahedral meshes). At each of these triangles or tetrahedra, numerical dissipation is the smallest when the exact edge is used. If the edge is pushed outward from the solution point by using a stretching parameter sigma and let $\sigma = 1$ denotes the exact location of the edge, then when sigma is increased, so is numerical dissipation. Controlling of sigma depends on the numerical dissipation desired and the shape of the element. UNICONES allows users to control numerical dissipation through the following parameters.

For most cases, a constant (scaled) numerical dissipation can be set by the following combination to begin with

```
alpha  1.  // can use 0.1 or smaller for low-speed flows or acoustic waves
beta   -2000
dissip_ctrl  1.e-12. // to turn its effects off
epsilon  2~10 // (use larger value for high-speed flows and with shocks)
```

In the above, sigma value is uniformly set to the epsilon value specified. For highly stretched viscous meshes, use the following combination:

```
alpha  1.  // can use 0.1 or smaller for low-speed flows or acoustic waves
beta   -200 // exponential stretching parameter
dissip_ctrl  -1.2 // minimum dissipation away from the wall near largest cell
epsilon  300  // maximum dissipation for highest aspect ratio near wall mesh
```

Note in the above the sigma value is changing from 300 to 1.2 with an exponential stretching factor of 200 (large value, faster variation away from wall). One

alternative to control numerical dissipation control is to scale dissipation based on local grid spacing like the following:

```
alpha  1.  // can use 0.1 or smaller for low-speed flows or acoustic waves
beta   -1.05 // the absolute value 1.05 is the minimum scaling factor
dissip_ctrl 20 // max. scaling factor at wall, as indicated by positive value
epsilon 1.5 // global minimum value for sigma (after scaling)
```

In general, sigma value should be large enough to maintain numerical stability and not allowing odd-even decoupling of the solution (high wave-number oscillations).

One important note to pay attention to is that the above dissipation control parameters, along with the time step or CFL number inputs for MPI runs are in fact an array input. The design of UNICONES is to allow each MPI block to have its own time step or numerical dissipation control. Users can use different values for different MPI blocks. For example, alpha = 1.0, 2.0, 1.2, 0.5 implies four different alpha values to be applied for a total of 4 blocks in the computation. In most situations, this choice cannot be easily made and the same value applies to all blocks. In these situations, a wild card can be used. For instance, if there are a total of 8 MPI blocks, one can do:

```
alpha          8*1.0
beta           8*-20000
dissip_ctrl    8*1.e-15
epsilon        8*1.5
dissip_per_cfl false
cfl            8*0.6
// or alternatively
// dt_const    8*0.002
```

Users need to make sure enough number of inputs in these arrays are specified in the input script or the code won't run properly.

=====

shock_gradp_value: If specified as a positive float, it will be used as the criterion for numerical dissipation control near the shock. When the normalized pressure gradient second derivative is greater than the value (implying within the shock), numerical dissipation specified by the following three parameters will be used. Otherwise, numerical dissipation specified by alpha, beta, epsilon, and dissip_ctrl will be used. If this number is negative, no shock filter will be done. Using this option allow uniformly small numerical dissipation to be used outside of the shock while a larger value at the shock guarantees stable time marching. Default is negative.

shockf_gradp_beta: beta value to be used only for shocks only.

shockf_gradp_eps: epsilon value to be used for shocks only.

shockf_gradp_dissip_ctrl: The dissip_ctrl value to be used for shocks only.

Typical use of the shock dissipation control is to apply a constant dissipation $\sigma = 8$ like the following:

```
shock_gradp_value 800.    // this value is case dependent
shockf_gradp_beta -2000
shockf_gradp_dissip_ctrl 1.e-14
shockf_gradp_eps 8.      // normally a value around 5 suffices
```

Also note that for all 2D runs with the built-in flow solvers, the second derivative of pressure is computed and written in the Tecplot™ output file “xtecpl_sol_allz.dat” as the variable “vort”. Users can first run the case without and pressure gradient shock capturing (negative `shock_gradp_value`), then plot the contours of the computed vort values in order to determine how to set the input parameter properly for shock capturing.

`Convg_norm_type`: Species choices of error norm type. Possible choices are L1NORM or L2NORM. Default is L2NORM.

Category 5: Instruction scripts related to turbulent flow computations

`turb_model_option`: Specified turbulence model type (laminar, sa_model, komega_wilcox, komega_sst, smagorinsky, and dsm_model, i.e., dynamic Smagorinsky model), default laminar.

`construct_wall_dist_db`: If set true, the wall distance database needed for RANS calculations will be generated and written to a file. For large size meshes, generating this database first can save computational time for later runs. Default is false.

`read_in_wall_dist_db`: If set true, the turbulence RANS model wall distance will be read in from file directly (generated after the first pre-processing run with `construct_wall_dist_db = true`). Default is false.

`skip_turb_when_dump_zones`: If set true, when the code is dumping zones, the turbulence part will be skipped. This is intended to be used to write RANS or LES solutions as a laminar case for later use. Default is false.

`skip_turb_when_read_zones`: If set true, when the code is reading in a restart or initial condition zone dump files, the turbulence part will be skipped. This is intended to be used to start or restart a RANS calculation from a previous laminar run. Often times, starting from laminar solutions would help quick convergence of RANS runs. Default is false.

`output_perturb_solution`: If set true, output perturbation quantities in the solution files, not the total variables (intended to be used with fixing mean flow options (see `mean_flow_zdum_fn`). Default is false.

`sugrid_turb_fluct_model`: if set true, a new subgrid-scale fluctuation based LES model will be triggered. Default is false.

`les_wall_model`: an integer of value 0 or 1. If set to 1, a simplified wall model based on projection of wall shear stress will be imposed in the LES runs. If set to 0, no wall model is imposed. Default is 0.

`les_vwall_slip_vel`: an integer of value 0 or 1. If set to 1, a slip velocity based on extrapolation will be imposed in the LES runs. If set to 0, no-slip conditions would be applied. Default is 0.

Note 2: =====

To compute viscous flows with a Reynolds Averaged Navier-Stokes (RANS) model, users need to first generate a viscous mesh with a y^+ value around 1 near the wall to ensure accuracy for most cases. For time accurate RANS computations, this small near-wall mesh results in a very small time-step; most, if not all CFD computations do not employ such a small mesh for unsteady RANS calculations due to efficiency considerations. Instead, a near wall mesh of around y^+ of 50 is used along with a wall function or wall model to replace the regular no-slip boundary conditions. As of now, a wall function is not implemented in UNICONES yet. It is therefore advised to have proper near wall mesh resolutions for RANS computations. To speed up convergence to steady state, users can run RANS with `constant_cfl = true` in the beginning. Time accurate computations can be run once the solution settles down. Alternatively, laminar flows can be run initially to let the flow develops, then turn the RANS computation by using the laminar solution as the initial conditions. In the input script, users turn on RANS by specifying `turb_model_option` (see above) to be one of the four models implemented (SA, k-omega SST, k-omega Wilcox, Smagorinsky, or dynamic Smagorinsky models). For the large-eddy simulation (LES) options, only time-accurate computations are meaningful. Users need to make sure eddies are properly resolved with good enough mesh resolutions. For 2D RANS, users just need to turn on the model by specifying `turb_model_option` with one of the models and then run the code until the boundary layer develops into the turbulent type and check the output file `turb_solu.dat` to make sure `uplus`, `yplus`, and eddy viscosity (`muT`) has been converged in the correct range and solutions have converged.

The freestream turbulence level can be adjusted if necessary. By default, for SA model, the freestream turbulence variable (normalized viscosity) is set as

```
const double what_fstream = 1.341946;
```

This freestream value can be adjusted by preparing a file called “SA_nu_fst.dat” in the running directory, in which the desired `what_fstream` is specified. Note that when the default value is desired, this file must be removed, otherwise the value in it will always be honored.

For k-omega SST model, the default values are:

```
const double k_fstream = 9.e-9;
const double omega_fstream = 1.e-6;
```

These two values are taken from CFL3D manual. They are normalized by acoustic speed at the freestream conditions instead of the freestream velocity. UNICONES renormalizes these values by the freestream velocity and displays them in the stdout when running. The normalization used by UNICONES is thus,

$$(k_{\infty}, \omega_{\infty}) = \left(\frac{k_{\infty}^*}{U_{\infty}^2}, \frac{\mu_{\infty} \omega_{\infty}^*}{\rho_{\infty} U_{\infty}^2} \right)$$

To change the default value, a file called “SST_k_omega_fst.dat” needs to be prepared and in it, freestream turbulence kinetic energy and eddy viscosity are specified. For example, a file contains the following line:

```
1.e-4 0.001
```

implies that

$$\frac{k_{\infty}^*}{U_{\infty}^2} = 0.0001 \quad \text{and} \quad \mu_T = \frac{k_{\infty}}{\omega_{\infty}} = 0.001$$

Similarly, when the default values are desired, the file “SST_k_omega_fst.dat” must be removed from the running directory.

For 3D RANS computations with the serial version of UNICONES using multithreading, just turn on the model, similar to the 2D RANS computations described above. However, if one intends to decompose the domains and use MPI version, one needs to first generate a wall-normal distance database as a pre-processing step (for both serial and MPI runs with multiple domains). To do so, follow the sequence below:

- a. Prepare the input script as desired (for MPI runs, prepare accordingly) and add the following lines in the input script:

```
turb_model_option komega_sst // or other model options
construct_wall_dist_db true
read_in_wall_dist_db false
num_iter 0
apply_pthread true // turn on multithread to speed up
max_nthread 1000 // set maximum 1000 threads
// Note: do not set num_zones_per_block for MPI runs
```

- b. Do the pre-processing step by running the sequential executable (not the MPI executable). Let's say the input script is call run_turbflow.in, do

```
./unicones -t .gen_wall_dist run_turbflow.in
```

If a node with many cores is available, this pre-processing run will be much more efficiently executed by setting apply_pthread inputs as above.

- c. Upon completion of the above sequential (multi-thread) run, a file called “Wall_dist_db.dat” should be generated. Then proceed to the regular MPI run by modifying the script as in the following:

```
turb_model_option komega_sst // or other model options
construct_wall_dist_db false
read_in_wall_dist_db true
num_iter 10000 // Change it back to desired numbers
apply_thread false // turn multithread off
// Note that each different domain decomposition will
// generate a wall-distance database differently,
// this preprocessing has to be done separately.
// In other words, Wall_dist_db.dat can't be shared.
```

Category 6: Instruction scripts for advanced features:

6.A: Immersed boundary method (IBM):

immersed_boundaries: whether to apply the immersed boundary method to a background mesh or not. Default is false (see below for a more detailed description)

`stationary_solid`: set this input to true if a stationary solid is of interest when `immersed_boundaries = true`, default is false. Set this if the solid in IBM does not move.

`rigid_moving_body`: if a moving rigid body is to be computed, set this to be true. Default is false. In this case, a motion DLL is needed, see below.

`SolidPhaseVarIndex`: the index number for the variable phase index (0 for fluid and 1 for solid). For flow solver, no need to specify this index. For user-defined GCL solver, must specify which index is used for the phase variable (for the built-in incompressible Navier-Stokes solver under the “`solve_gcl = true`” option, this is 4 for 2D and 5 for 3D flows).

`SolidPhasePhiMin`: this real number is used to determine whether a cell is a solid or fluid. If the phase variable (which varies between 0 and 1) exceeds this threshold, the cell is considered a solid cell. Default value is 0.5.

`SolidPhaseMotionDLL`: this string stores the DLL file name for the 6dof motion of the solid. Codes inside this DLL is used to calculate current velocity of a cell based on previous velocity, time, and time step passed in. Users can integrate the 6dof motion description into this DLL for UNICONES to determine next velocity vector for the movement. Default is an empty string, which means a fixed (zero velocity) solid.

`SolidPhaseAlpha`: this real number is used to calculate numerical dissipation for the phase equation when IBM is used. This parameter is only used for the phase equation. Default value is 1.0

`SolidPhaseEpsilon`: this real number is used as the numerical dissipation parameter sigma value (see descriptions of parameter epsilon above). If left unspecified (default value is 0), no special numerical dissipation control is applied to the phase equation in the IBM approach. In general, numerical dissipation can be smaller for the phase equation as compared to other fluid conservation equations. A value between 1.01 and 1.1 can be used to reduce dissipation errors near the solid phase boundaries.

`SolidPhaseMuRatio`: this real number is the ratio of viscosity between the solid/liquid to the ambient air. If only a solid is involved, this number is irrelevant because the motion of the solid has nothing to do with the solid viscosity. However, for two-phase flow, this ratio sets the viscosity of the liquid relative to the ambient air. Default value is 1.0.

`Weber`: if this real number is non-zero (default is 0.), surface tension is imposed at the liquid/air interfaces. For solid IBM, this number is irrelevant. Default value is 0 (no surface tension is accounted for).

3. Preparation of Mesh and Boundary Condition Files and Initialization of Flow Field

3.1 Procedure for exporting mesh and boundary condition files from Pointwise® after the mesh is ready:

1. Click “CAE” panel at the top, and then select “Set Dimension” to be “2D” or “3D”
2. Click “Grid” panel at the top, and then select “Set Type” to be “Unstructured”
3. Click “CAE” panel at the top, and then “Select Solver”, a list will show up and choose “STAR-CD”
4. Click “Domain” in the left panel or use mouse to crop the entire domain to select the entire domain
5. Click “CAE” panel at the top, and then select “Set Boundary Conditions”, a panel to the left will appear.
6. Click and select every boundary below “Unspecified”, and then click “Delete” button to remove all default set up
7. Click “New” button to create new boundary condition. If there are 4 boundary conditions to be used for the entire domain, then click the “New” button 4 times, vice versa
8. If there are more than one type of inlet boundary conditions or no appropriate boundary flag provided by Pointwise®, users can select any boundary conditions (such as stagnation “STAG”) and modify them later to those available in UNICONES that suite your need (Section 3.2 below).
9. Set up the name and type of those newly created boundary condition
10. Select each boundary of the mesh, and then click the leftmost button to link each boundary to the corresponding boundary condition
11. Once the condition for each boundary is properly set up, click “Close” button in the left panel to exit the boundary condition setup
12. Repeat Step 4 to select the entire domain, and then click “File” panel at the top, select “Export”, and then “CAE”, a dialog panel will appear for you to specify the name for all the files associated with STAR-CD®. Only one name needs to be entered, and a set of files will be created with different file extensions (“.vrt”, “.cel”, “.bnd”).
13. If you have not saved the layout/setup file for your current session, you can click “File” panel at the top, select “Save As” (if a new file to be created) or “Save” (if an existing file to be over-written) to save the Pointwise® working file.

3.2 Procedure to convert those files exported from STAR-CD to the mesh and boundary condition files in UNICONES format:

1. Go to the working directory/folder where the STAR-CD mesh and boundary condition files are located
2. If the wall boundary condition is isothermal (fixed temperature), then change the “WALL” flag in STAR-CD boundary condition file (“.bnd”) to “isothermal” since the default condition for “WALL” in UNICONES is adiabatic.
3. The exit boundary with the “OUTL” flag is treated as a non-reflecting boundary in UNICONES. If a known set of flow conditions to be applied at the exit boundary, then replace the “OUTL” flag in the “.bnd” file with “nozzle” flag.

4. If there are more than one type of inlet boundary condition or the freestream boundary condition (all flow variables are fixed) is not suitable for the problem of interest, then the description of using DLL to specify inlet boundary condition is provided in Step 6 shown below.
5. Type “getPGrid” at the command prompt and follow the Q&A process to create UNICONES mesh and boundary condition files. A sample process is shown below, in which red italic fonts are user’s input:

```

=====
Using starcd vrt and cel files? y

Enter starcd vertex file name = test.vrt

Enter starcd cell file name = test.cel

Enter the bc file in starCD format: test.bnd

Is this a 3D grid? (y/n): n
Re-orient the mesh? (y/n): n

Writing out grid+bc files in unicones format? (y/n): y

Writing unicones grid in mixed_mesh format? (y/n): n

Enter the output grid file name: test.grid

Enter the output bc_file name: test.bc

Read in 201201 vertices coordinates
Read in 400000 elements
  This mesh is not a mixed mesh
  mesh type is not an unstruct48_ mesh
bnodes_db size = 1200000
bound_nodes_ size = 2400
bound_faces_ size = 2400
Summary of elements:
  number of vertices = 201201
  number of elements = 400000
  number of boundary elements = 2400
Read in 2400 boundary condition flags
=====

```

6. From Step 5, two boundary condition files will be created, “test.bc” and “test.bc.def”. If special boundary conditions were used in Step 2, edit “test.bc.def” file and change the values associated with the boundary condition. For example,
 - a. For the isothermal boundary, the line with its 2nd column has a value of 3. Add a value which is equal to the non-dimensional wall temperature (normalized with the freestream temperature) after the 2nd column.
 - b. If a fixed pressure/velocity or freestream condition to be used at the “Exit” boundary which has been changed to “nozzle” as shown in Step 3, then edit the line with its 2nd column has a value of 50 and replace its following values, “292.506 -28.695 0.00 133.225”, with a set of appropriate values, such

as “1. 1. 0. 5.46”, which are $\rho, \rho u, \rho v, \rho e_t$ for 2-D flow, or as “1. 1. 0. 0. 5.46”, which are $\rho, \rho u, \rho v, \rho w, \rho e_t$ for 3-D flow normalized with the freestream density and velocity, respectively. If DLL was used for the “OUTL” boundary, then edit the line with its 2nd column has a value of 11 and add the name of DLL, such as “./exit_bc.so”, after the 2nd column.

- c. If DLL was used for the inlet boundary, then edit the line with its 2nd column has a value of 11 and add the name of DLL, such as “./inlet_bc.so”, after the 2nd column.
- d. Noted that users are advised to modify “.bc.def” file instead of the “.bnd” file as the boundary condition flags revised by users may not be supported by “getPGrid” pre-processor and leads to unnecessary errors. The available boundary condition types are listed in Appendix E. Users are encouraged to understand them before applying them appropriately.

3.3 Procedure to decompose the computational domain into desired number of numerical blocks, if parallel computing with MPI will be employed for UNICONES:

1. If serial computing with p-thread will be performed, then users can skip this process.
2. Go to the directory/folder where the UNICONES mesh and boundary condition files are located
3. Enter “bDecompose” at the command prompt and follow the Q&A process to create the partitioned mesh and boundary condition files. A sample process is shown below, in which red italic fonts are user’s input:

```
=====
Input unicones grid file name: test.grid

Input unicones bc file name: test.bc

Is this a quad or hex mesh? y

Is this grid in mixed_mesh format? n

Is this a 3D grid? n

Rescale the coordinates? n

Re-orient the mesh (only works for non-mixed meshes)? (y/n): n

decompose domains using weighted partitioning (y/n)? n

Decompose into how many partitions? 4

Output file name: test_mpi

signed area max = 0.000112546 min = 2.5e-05
Read in unstructured grid
  No. of elements = 14400 No. of vertices = 14641
  No. of boundary elements = 480
bc segments definition file is cavity48.bc.def.mpi
finished reading bc file
Done reading one-block grid & bc files
```

```

Total number of vertices for connectivity = 57600
total vertex count = 57600
METIS return status = 1
metis partition completed
Summary of domain: 0
  number of vertices = 3737
  number of elements = 3607
  number of boundary elements = 258
Summary of domain: 1
  number of vertices = 3730
  number of elements = 3603
  number of boundary elements = 252
Summary of domain: 2
  number of vertices = 3740
  number of elements = 3607
  number of boundary elements = 264
Summary of domain: 3
  number of vertices = 3706
  number of elements = 3583
  number of boundary elements = 244
=====

```

3.4 Procedure to specify initial flow field condition

There are 3 options to initialize flow field:

1. In the input script file, specify `init_cond_depQ = 1 1 0 0 0.524`, where the values of Q vector are described in Section 2.2
2. Use a solution from previous run (for which you get a zone dump file, say, `binDumpZ.dat.run1`, then do

```
init_cond_use_zdump_fn      "binDumpZ.dat.run1"
```

3. Use DLL, write your DLL in C++ as in the sample file (`init_cond.cc`) shown in Appendix C, then compile it like

```
g++ -fPIC -shared -o init_cond.so init_cond.cc
```

or use `icpc` in lieu of `g++`, then add:

```
init_cond_use_zdump_fn      "./init_cond.so"
init_cond_use_zdump_fn_usedll      true
```

This will instruct UNICONES to apply the DLL as the initial conditions. Users are free to define initial condition based on given coordinates (x, y, z).

4.0 Generating and Post-Processing Data Files

4.1 Generating property values on the wall surface from a restart zone dump file:

If the zone dump file was generated from a serial run with p-thread, then set `apply_pthread` to `false`, and `setnum_iter = 0` in a restart input script file. Re-run UNICONES and a file named `"vsurface_qty.dat.##"` will be created, where `"##"` is the casetag specified in the run script file. If an adiabatic wall boundary condition was specified, then the file contains 5 variables: x, y, p, T, C_f . If an isothermal wall boundary condition was specified, then the file contains 13 variables: $x, y, x_1, y_1, p, T_1, dy, q_w, dtdy_2, dtdy, C_h, qf_2, C_f$, where subscript "1" denote the value at the point next to the wall.

4.2 Procedure to post-process the output files of interest:

1. Post-process flow field using Tecplot™:
 - a. Rename the Tecplot™ data file generated by UNICONES (typically has a name of `"xtecp_sol_allz.dat.##"` where `"##"` is the casetag specified in the run script file) to any desired name, such as `"tecplot.dat"`
 - b. Type `"preplot tecplot.dat"` to convert the Tecplot™ formatted ASCII file to Tecplot™ formatted binary file (`"tecplot.plt"`) to reduce the file size and delete `"tecplot.dat"` file.
 - c. Run Tecplot™ (`"tec360"`) and load `"tecplot.plt"` into Tecplot™ for post-processing
2. To extract data generated from `"probe_points_def_fn"` in UNICONES:
 - a. In your working directory/folder, there should be a file created by UNICONES, `"probe_data_dist"`, and a file containing the coordinate of the locations for data probing defined by the user in the input parameter `"probe_points_def_fn"`, e.g., `"probe.dat"`
 - b. Type `"readpd"` and follow the Q&A process. An example is shown below:

```
=====
Processing multiple probe data output files (y/n)? n

Input the probe_data_dist file name: probe_data_dist

Input the probe points file name: probe.dat

Is this a 3D file? n

Enter the point index: 0

Enter the output file name: pdata-1.dat
=====
```

Note: The point index refers to the sequence of data probing location, where "0" is for the 1st location. Red fonts are user's input

3. For calculating the frequency using Fast Fourier transformation:

- a. In your working directory/folder, there should be a file or a set of files created from “readpd”, depending on the number of locations the user has extracted the time dependent data
- b. Type “probe_fft” and follow the Q&A process to compute the frequency for each set of data. An example is shown below:

```
=====
Input the probe data file name: pdata-1.dat

time from 128 383.997
Input time intervals for FFT (y/n)? n

Input normalization parameter for time (to convert into second):
3.0e-7

00 = (0.902911,0) (0.184592,0) (0.000927829,0) (0,0) (1.07893,0)
(1.08744,0) (0.166553,0)
mean values = 0.902911 0.184592 0.000927829 0 1.07893 1.08744
0.166553
mean v = 0.000925335 0.0115449
mean T = 1.07893 1.16414
mean p = 1.08744 1.18391
=====
```

Note: Red fonts are user’s input. The value of the 3rd input is to convert non-dimensional time in the data file to a dimensional time. The value can be obtained from the standard output file of UNICONES, “case.out” by dividing the value of the reference length scale “Lscale” by the value of the reference velocity “u” (i.e., Lscale/u), where they can be found below the line “Input flow-related normalization parameters”. A file, “Probe_spectrum.dat”, will be generated from this process. If you have other data files to be post-processed by “probe_fft”, then rename the output file to other name before re-running “probe_fft”.

For MPI runs, the input probe points may be located in many different MPI ranks (or blocks). Hence instantaneous probe point solutions are written to multiple files, with the number of files depending on how many ranks or blocks are in the domain decomposition. Post-processing of the MPI probe data is different from the above sequential runs. Let’s assume we use a probe points definition file called my_probe_loc.dat and the MPI run was carried out with a tag called “.runx1”, then similar to the sequential run, we add a line probe_points_def_fn = “my_probe_loc.dat”. Upon finishing, a number of files are generated: probe_data_distz0,probe_data_distz1,probe_data_distz2,.....probe_data_distz127 (assuming there are 128 ranks in the MPI run), which contain the distance information needed for post-processing. In the meantime, the binary probe solution data is written to: probe_dataz0.runx1, probe_dataz1.runx1,probe_dataz2.runx1,...probe_dataz127.runx1. To post-process the multiple probe data files, we need to collect all these data

files into an information file (let's called it `all_probe_files` that contains the following:

```
=====
my_probe_loc.dat
probe_data_distz0
probe_data_distz1
probe_data_distz2
probe_data_distz3
probe_data_distz6
probe_data_distz7
probe_data_distz8
probe_data_distz9
probe_data_distz10
...
probe_data_distz125
probe_data_distz126
probe_data_distz127
=====
```

Now, we can do post processing readpd with the following procedure:

```
=====
Processing multiple probe data output files (y/n) ? y
```

```
Input the multiple-file information file name :
all_probe_files
```

```
Is this a 3D file ? y
```

```
Enter the point index (-1 for all points) : 3
```

```
Enter the output file name : probe3_time.dat
```

```
Input additional node indices (as in probe_points file
specified in the information file)
```

```
time sequence will be written to pn.dat where n =
index)
```

```
input -99 to end the input
```

```
2
```

```
4
```

```
-99
```

```
i = 0 file = probe_data_distz0
```

```
probe data output in file : probe_dataz0.run30_probe
```

```
Read in 720458 points in the probe_data_dist file :
```

```
probe_data_distz0
```

```
i = 1 file = probe_data_distz1
```

```
probe data output in file : probe_dataz1.run30_probe
```

```
Read in 720458 points in the probe_data_dist file :
```

```
probe_data_distz1
```

```
i = 2 file = probe_data_distz2
```

```
probe data output in file : probe_dataz2.run30_probe
```

```

    Read in 720458 points in the probe_data_dist file :
probe_data_distz2
i = 3 file = probe_data_distz3
...

```

Upon finishing, instantaneous probe data is written to p3.dat, p2.dat, p4.dat. If users answered -1 to the point index question (and answered 0 for the number of grid files question, solutions for all probe points will be outputted in a file called probe_points_sol0.dat if only one time step was run (if multiple time steps, there will be probe_points_sol1.dat, probe_points_sol2.dat ... etc.) .

Note that when preparing the file all_probe_files, only the existing probe data output files can be specified. Since probe points may not spread over all MPI blocks, only select those ranks with output written. To safely prepare this information file, the following Python script is used as an example:

```

import string
import os

# process probe data generated by MPI runs

#tag = "run30_probe"
#probe_pts_fn = "probe_pts_sr72stl.dat"
tot_rank = 1120

probe_dist_out_fn = "all_probe_files"
out = open(probe_dist_out_fn, "w")
out.write(probe_pts_fn + "\n")

strpb = "probe_data_distz"
print('Processing MPI generated probe files...\n')

for i in range(tot_rank):
    stri = strpb + str(i)
    # write the file name if exists
    if os.path.isfile(stri):
        out.write(stri + "\n")

out.close()
print('probe data summary file to be used for readpd written in ',
      probe_dist_out_fn)

```

In cases of immersed boundary method runs, it is usually necessary to do post-processing in order to extract viscous surface solutions. For 2D IBM runs, surface definition is simply a collection of wall points. Users can just put all surface coordinates (one point a line) as the probe_points_def_fn, then run one time step. Using the above processing by choosing the -1 option to the point index question, then surface solutions will be written out in probe_points_sol0.dat. For 3D MPI runs, only writing out discrete points are not enough to visualize the results, hence

users need to prepare a surface definition file. One good way to handle this is to prepare an STL file for the surface. First convert this STL file into a Tecplot format (**there is a code to do this, the ray-tracing code reader**) in the 3D vertex format and then extract all vertex points coordinates in the probe definition file. Upon finishing the one time step UNICONES run, post process the probe files similar to the above. The STL Tecplot file should take the following format:

```
=====
Title = " STL file using Tecplot format"
variables = "x", "y", "z"
zone Nodes = 720458 Elements = 1440920 ZONETYPE= FETriangle,
DATAPACKING=POINT
  0.0000000E+00  0.0000000E+00  0.0000000E+00
  8.5827638E-04  5.8988393E-05 -1.7767078E-04
  8.5827638E-04  5.8988393E-05  1.7767078E-04
  1.4822236E-03  4.5081423E-04 -1.1844796E-04
  1.4822236E-03  4.5081423E-04  1.1844796E-04
  1.7108950E-03  3.1855862E-04 -2.6757785E-04
  1.7108950E-03  3.1855862E-04  2.6757785E-04
  ...
  ...
  1.3081997E+01  3.4694470E-18  0.0000000E+00
  1.3082082E+01 -3.4832144E-03  0.0000000E+00
476993 477995 476991
479060 478520 476993
476993 478520 477995
479060 479059 478520
477995 478340 476991
478520 478340 477995
479060 480104 479059
480105 480104 479060
479059 479058 478520
...
=====
```

The processing records by running readpd look like the following:

```
=====
myhost:/work1/unicones_runs$ ./readpd
Processing multiple probe data output files (y/n) ? y

Input the multiple-file information file name : all_probe_files

Is this a 3D file ? y

Enter the point index (-1 for all points) : -1

Number of grid files = 1

Enter Tecplot grid file(s) = sr72_stl_tec.dat

Enter the output file name : sr72_surface_solu.dat

i = 0 file = probe_data_distz0
  probe data output in file : probe_dataz0.run30_probe
  Read in 720458 points in the probe_data_dist file :
probe_data_distz0
=====
```

```
i = 1 file = probe_data_distz1
  probe data output in file : probe_dataz1.run30_probe
  Read in 720458 points in the probe_data_dist file :
probe_data_distz1
i = 2 file = probe_data_distz2
  probe data output in file : probe_dataz2.run30_probe
  Read in 720458 points in the probe_data_dist file :
probe_data_distz2
i = 3 file = probe_data_distz3
  probe data output in file : probe_dataz3.run30_probe
  Read in 720458 points in the probe_data_dist file :
probe_data_distz3
i = 4 file = probe_data_distz6
...
...
=====
```

Appendix A

Sample Input Files

A sample input file for serial computing with p-thread is shown below:

```
=====
//
// Double cone 25-55 sharp cone
//

DomainType          e2d

apply_pthread        true
num_zones_per_block  50 // up to 2*no. of cores
max_nthread          54 // num_zones_per_block + 4 or 5

// for the first run, use false, after ZIface_db.* have been generated,
// use true to save time
    readin_iface_info  false
//mpi_comm_map_fn = "cav_comm_map.dat"

//for large memory cases uncomment these 2 lines
//mpi_use_lean_mem  true
//mpi_dump_each_rank  true

    restart            false    //true for restating old run
//restart file for convergence from previous run
    convg_fn_restart   "unicones.enorm.r2"
//restart file containing data from previous run
    zdump_fn_restart   "binDumpZ.dat.r1"
//file for initializing run with a different flowfield
    init_cond_use_zdump_fn  "binDumpZ.dat.r1"

grid_fname           "test.grid"
bc_fname              "test.bc"

grid_type             unstructured
flow_solu_type        ns_SOL
axisymmetric_2D       true

Transp_prop_option    CONST_PRANDTL
fs_Mach               12.65
fs_Tinf               99.38
fs_UnitRe             2.2853e+5 // per meter
Linf                  0.0254    //to convert mesh to meter
Prandtl               0.72
Rgas                  296.93

// initial conditions as conservative variables
// non-dimensionalized by fs_Tinf, Uinf, rho_inf
//init_cond_depQ       1 0 0 0.55

// Probe data
//probe_points_def_fn  "probe_file_name"
```

```

//probe_output_skip_niter          5
//calc_time_mean_values            true
//calculate mean flow details

use_incenter            true
//comment use_incenter if using quad/hex mesh as incenter is undefined

alpha                    2
beta                     -5000
dissip_ctrl              1.e-12
epsilon                  10.0 //for Local time stepping use >3
dissip_per_cfl           false

shock_gradp_value = -2000 // cut of value for shock detection
shockf_gradp_beta = -2000
shockf_gradp_dissip_ctrl = 1.e-14
shockf_gradp_eps = 3.0

cfl                      0.1

//steady state
//constant_cfl           true

//time accurate
//constant_cfl           false
//dt_const               0.01

//talts run
unsteady_lstep           true

num_iter                  10000
print_convg_per_niter    100
print_sol_per_niter      500
dump_zone_per_niter      100

write_wall_force          true

```

A sample input file for parallel computing with MPI is shown below:

```

solve_gcl                true
DomainType                e3d
axisymmetric_2D           false

mpi_comm_map_fn            "mpi_comm_map.dat"
block_mpi                  true
readin_iface_info         true

mpi_use_lean_mem           true
mpi_dump_each_rank         true

grid_type                  unstructured
grid_data_format           c_binary
multi_block                true
grid_fname                 "mpi_grid.dat"
bc_fname                   "mpi_bc559.dat"

```

bc_file_use_table	true
restart	false
zdump_fn_restart	"run1.559"
convg_fn_restart	"unicones.enorm.run1"
fs_Mach	0.2
fs_Tinf	273.
fs_UnitRe	391953
Linf	1.0
Transp_prop_option	CONST_PRANDTL
Prandtl	0.72
init_cond_depQ	1 1 0 0 0
alpha	560*1.0
beta	560*-20000
dissip_ctrl	560*1.e-15
epsilon	560*2.0
dissip_per_cfl	false
cfl	560*0.6
constant_cfl	false
unsteady_lstep	true
use_incenter	true
num_iter	2000
print_convg_per_niter	2000
print_sol_per_niter	2000
dump_zone_per_niter	5
write_wall_force	true

=====

Appendix B

The immersed boundary method is implemented by solving the phase equation as

$$\frac{\partial \phi}{\partial t} + \vec{V} \cdot \nabla \phi = 0$$

in addition to the governing Navier-Stokes equations. The instantaneous velocity vector is indicated by $\vec{V}$ in the above equation. The phase variable ϕ is set to be 0 for the fluid and 1 for either the solid or liquid. For a stationary solid, $\phi = 1$ and $\vec{V} = 0$. For two-phase flows, the phase variable can be varied from 0 to 1 and the instantaneous velocity is not necessarily zero. Let the instantaneous velocity vector be denoted by $\vec{V} = (\tilde{u}, \tilde{v}, \tilde{w})$ and density by ρ . For the solid or liquid phase, a source term is added to the momentum equation via

$$S = \frac{1}{\Delta t} \begin{bmatrix} \rho \tilde{u} - \rho u \\ \rho \tilde{v} - \rho v \\ \rho \tilde{w} - \rho w \end{bmatrix}$$

where Δt is the marching time step. The above source term would force the velocity solution in the solid region to be $\vec{V}$ as desired. This source term augmentation approach is a common practice for most IBM methods. If a body force is present or the acceleration of the solid (liquid) is obtained via a six degree of freedom dynamics equation, the instantaneous velocity needs to be updated at each marching time step. This effect can be captured by the equation of motion implemented as

$$\vec{V}^{n+1} = \vec{V}^n + \vec{a} \Delta t$$

A body force term is automatically added to the momentum equation through an acceleration vector $\vec{a}$. This approach can also be used to add gravitational forces for a two-phase flow calculation.

In the current implementation of IBM, each solution element is assumed to have a unique function of the phase variable that varies as a first-order Taylor series expansion, similar to all other flow variables. It is therefore not necessary to use interpolation to find out the point of intersection at the solid boundaries. Strong conservation law integration of the CESE method in the space-time domain would ensure local as well as global flux conservation based on valid region of the solution element. When the grid is refined, the solution accuracy near the solid boundaries would continue to increase until it converges with mesh refinement. This approach avoids the tedious interpolation step to find out wall intersections in common IBM implementations using either finite-difference or finite-element approaches.

Sample DLL to initialize flow field for IBM

```
=====
void
initQ_at_p(double x, double y, double z,
           double* Q, double* dQdt,
```

```

        double* dQdx, double* dQdy, double* dQdz) {

const int nvar = 4; // for 2D incompressible N-S IB solver
double p = 1;
double u = 1; // if desired to start from quiescent set it to 0
double v = 0;
double phi = 0; // fluid

double xc1 = 0, yc1 = 0;
double rad = 2.;
double R = sqrt((x-xc1)*(x-xc1) + (y-yc1)*(y-yc1));
if( R <= rad) {
    phi = 1.00; // solid
    u = 0;
    v = 0;
    p = 1;
}

Q[0] = p;
Q[1] = u;
Q[2] = v;
Q[3] = phi;

// derivatives not very important
for(int i = 0; i < nvar; ++i) {
    dQdt[i] = 0;
    dQdx[i] = 0;
    dQdy[i] = 0;
}
}

// Or for compressible built-in flow solver

void
initQ_at_p(double x, double y, double z,
            double* Q, double* dQdt,
            double* dQdx, double* dQdy, double* dQdz) {

const int nvar = 5;
const double mach = 4;
const double gamma = 1.4;
double rho = 1;
double T = 1;
double u = 0;
double v = 0;
double phi = 0;
// use an initially uniform pressure and let solid develop its on pressure
// through interaction with the fluid
double p = rho * T / (gamma * mach * mach);

double xc1 = 0, yc1 = 0;
double rad = 2.;
double R = sqrt((x-xc1)*(x-xc1) + (y-yc1)*(y-yc1));
if( R <= rad) {
    phi = 1.00;
    u = 0;

```

```

    v = 0;
    T = 1;
    rho = 50;    // can use an arbitrarily higher density for the solid
}

Q[0] = rho;
Q[1] = rho * u;
Q[2] = rho * v;
Q[3] = p / (gamma - 1) + 0.5 * rho * (u * u + v * v);
Q[4] = phi;

// derivatives not very important
for(int i = 0; i < nvar; ++i) {
    dQdt[i] = 0;
    dQdx[i] = 0;
    dQdy[i] = 0;
}
}

```

To allow rigid solid or liquid phase flow movement, users need to write a motion DLL like the following:

```

void get_6dof_vel(double time, double dt, double* p, double* center, double*
force, double* torque,
    double* U, double* V, double* W, double* phi) {

    const double radius = 2;
    // current velocities are passed on for integration with force
    // All quantities in this routine are based on normalizations specified
    // in the input script (see above)
    // Passing in, phi is the current phase variable value
    // pointer p points to (x, y, z) coordinates of the solution point
    // as x = p[0], y = p[1], and z = p[2] if 3D
    // Users can use the coordinate to determine motion separately
    // Region 1, set the velocity vector to any value as desired
    if( p[0] <= 0.1 && p[0] >= -0.1 ) {
        *U = -0.3;
        *V = 0.2;
        *W = 0;    // constant velocity movement
    }
    return;
}

// alternatives to integrate velocity using known acceleration
// Note: this acceleration would create an equivalent body force in
// the momentum equations
if( x <= 0.5 ) {
    // 6dof based on force and torque passing in from the solver
    double u0 = *U;
    double v0 = *V;
    double w0 = *W;

    double mass = 1.0;    // must know mass
    // if the acceleration is know, just set it directly, like:
    // double ax = 0, ay = -0.2, az = 0;
}

```

```

// if the force is calculated and passed over to this DLL
double ax = force[0] / mass;
double ay = force[1] / mass;
double az = force[2] / mass;

U[0] = u0 + ax * dt;
V[0] = v0 + ay * dt;
W[0] = w0 + az * dt;
// still need to factor in torque
return;
}
// solid spins around z-axis otherwise

double pi = acos(-1.0);
double rad = sqrt(p[0]*p[0] + p[1]*p[1]); // use (x, y) only
double ang = atan2(p[1], p[0]);
double Vel = rad / radius;
*U = -Vel * sin(ang);
*V = Vel * cos(ang);
*W = 0; // 2D

return;
}

```

As such, users can apply a range of different motion description for the given mesh. If simple logic based on coordinates do not work, users can write a more sophisticated logic in the above file using other data to help determine proper velocity vector.

With above DLL files ready, one need to prepare a baseline grid. This grid can be any grid as desired without any solid geometries specified. A good background mesh would be an isotropic triangular or tetrahedral mesh with proper grid resolution near where the flow would develop around the solid specified in the initial condition DLL. By using non-reflecting boundary conditions, the outer boundary need not to be very far away from the body. A typical input script looks like below.

```

DomainType      e3d

restart          false
convg_fn_restart "unicones.enorm"
zdump_fn_restart "binDumpZ.dat"
immersed_boundaries true
stationary_solid true // if stationary
/*
rigid_moving_body true // if the solid body is moving
SolidPhaseMotionDLL "./motion_translation.so". // must specify this
*/
/* for two phase flows, must not set either stationary_solid or
rigid_moving_body to be true, for instance,
Immersed_boundaries. True
SolidPhaseMotionDLL "./motion_gravity_new.so"
SolidPhaseAlpha 0.001

```

```

SolidPhaseEpsilon 1.05
SolidPhaseMuRatio 100
Weber 50
*/

SolidPhasePhiMin 0.001
SolidPhaseAlpha 0.01
SolidPhaseEpsilon 1.01 // use a lower dissipation for the phi equation
SolidPhaseMuRatio. 1.e-16 // solid does not have viscosity

grid_fname      "./grid/cubel_tet.g"
bc_fname        "./grid/cubel_tet_rot.bc"

grid_type        unstructured
flow_solu_type    ns_SOL

Transp_prop_option CONST_PRANDTL
Linf             1
fs_mach          0.2
fs_Tinf          300
fs_UnitRe        1000

// use full path to avoid resetting LD_LIBRARY_PATH
init_cond_use_zdump_fn  "./init_cubef_spinR.so"
init_cond_use_zdump_fn_usedll  true

alpha 1.0
beta -2000
dissip_ctrl 1.e-15
epsilon 1.1

apply_pthread      true
max_nthread        36
num_zones_per_block      32

use_incenter      true
dissip_per_cfl    false

dt_const          0.003
constant_cfl      false
num_iter          4000

print_sol_per_niter      200
dump_zone_per_niter      200
print_convg_per_niter    50
=====

```

If the initial geometry definition of the solid is simple, it is pretty straightforward to program the configurations in the DLL as described above. For more complex geometries with a given STL file, there is a pre-processing code to read in the STL file, define the bounding box, and then mark any point inside the bounding box with proper phase variables using the ray-tracing technique. Once the bounding box is available, any Cartesian or other background mesh to be used in the IBM calculations can be easily interpolated to the correct phase variable distributions in the initial

condition DLL file. If desired, the pre-processor can also be used to group the complex configurations into different parts that would move with different velocity or acceleration in the motion DLL file. These pre-processing codes are available for users to modify and adopt to different applications.

Appendix C

Sample DLL to initialize flow field

```
=====
//
//  initial conditions for a normal shock (Schardin problem)
//
#include <math.h>
#include <iostream>

using std::cout;
using std::endl;

// use C name mangling to avoid cross compiler issues
// given solution point coordinates, returns
// dependent variables vector
// Q = (rho, rho u, rho v, rho w, e)
// dQdx, dQdy, dQdz, dQdt are vectors of 4(2d) or 5(3d)
// each pointer points to an array of 4 or 5 elements
extern "C" void initQ_at_p(double x, double y, double z,
                          double* Q, double* dQdt,
                          double* dQdx, double* dQdy, double* dQdz);

void
initQ_at_p(double x, double y, double z,
           double* Q, double* dQdt,
           double* dQdx, double* dQdy, double* dQdz) {

    // 2D
    int nvar = 4;

    const double pi = acos(-1.0);
    const double gamma = 1.4;
    const double gml = gamma - 1.;
    const double mach = 0.44865;
    const double T1 = 347.844;
    double u1 = mach * sqrt(gamma * 287. * T1);
    double p = 1. / (gamma * mach * mach);
    double M2 = 0.;
    double T2T1 = 1./1.216;
    double r2r1 = 1./1.585;

    double u2 = M2 * sqrt(T2T1) / mach;
    double rho2 = r2r1;
    double p2 = rho2 * T2T1 / (gamma * mach * mach);

    if( x < 0. ) {
        // pre-shock conditions
        Q[0] = 1.0;
        Q[1] = 1.0;
        Q[2] = 0.;
        Q[3] = p / (gamma - 1.) + 0.5;
    }
}
```

```

else {
    Q[0] = rho2;
    Q[1] = 0.;
    Q[2] = 0.;
    Q[3] = p2 / (gamma - 1.);
}

dQdx[0] = 0;
dQdy[0] = 0;
dQdx[1] = 0;
dQdy[1] = 0;
dQdx[2] = 0;
dQdy[2] = 0;
dQdx[3] = 0;
dQdy[3] = 0;

}
=====

```

Appendix D

Sample DLL to Specify Boundary Condition

```
=====
//
//  prototype for dll for boundary conditions
//

#include <math.h>
#include <iostream>
#include <string>
#include <deque>
#include <vector>
#include <sstream>
#include <fstream>

using std::cout;
using std::cerr;
using std::endl;
using std::string;
using std::deque;
using std::vector;
using std::istringstream;
using std::ifstream;
using std::ofstream;

// compile this dll with "icpc -fPIC -shared -o inlet_bc.so inlet_bc.cc

// use C name mangling to avoid cross compiler issues
// given solution point coordinates, returns
// dependent variables vector
// Q = (rho, rho u, rho v, rho w, e)
// dQdx, dQdy, dQdz, dQdt are vectors of 4(2d) or 5(3d)
// each pointer points to an array of 4 or 5 elements
//
// On input, Q, dQdt, dQdx, .. etc contains the connecting element's solution
// at previous time step, if not used, just overwrite it with new
// boundary condition values
extern "C" void getQ_inst_dll(double t, double x, double y, double z,
                             double* Q, double* dQdt,
                             double* dQdx, double* dQdy, double* dQdz);

void
getQ_inst_dll(double t, double x, double y, double z,
              double* Q, double* dQdt,
              double* dQdx, double* dQdy, double* dQdz) {

    // free-stream conditions are at Mach 2.6
    // all quantities normalized by
    /*
    Normalization parameters:
    Mach = 2.6 Lscale = 0.0254
    T = 138.18, Re = 6.5577e6/m
    the following from ez4d stdout

```

```

*/
const double mach = 9.81;
const double u_inf = 1376.18;
const double rho_inf = 0.00945297;
const double T_inf = 48.978;

// nozzle stagnation conditions
const double gamma = 1.4;
const double t0 = 298.;
double time0 = 0; // starting time for the stag. pressure function below
const double m0 = 0.1465; // mach number

// time varying stagnation pressure
double p_end = 3.401; // in atm
double p_start = p_end * 0.01;
double p0t = p_start + (p_end - p_start) / 5. * (t - time0);
double p0 = fmin(p0t, p_end) * 101325.;

// compute static conditions
double ratio = 1. + 0.5 * (gamma - 1) * m0 * m0;
double ti = t0 / ratio;
double pi = p0 / pow(ratio, gamma / (gamma - 1.));
double ui = -m0 * sqrt(gamma * 287. * ti);

// non-dimensional values
double pN = pi / (rho_inf * u_inf * u_inf);
double TN = ti / T_inf;
double uN = ui / u_inf;
double vN = 0;

double rhoN = gamma * mach * mach * pN / TN;

Q[0] = rhoN;
Q[1] = rhoN * uN;
Q[2] = rhoN * vN;
Q[3] = pN / (gamma - 1) + 0.5 * rhoN * (uN * uN + vN * vN);

// set all derivatives zero
for(int k = 0; k < 4; ++k) {
    dQdx[k] = 0;
    dQdy[k] = 0;
    dQdt[k] = 0;
}
}
=====

```

Appendix E

UNICONES Mesh and Boundary Condition File Format

UNICONES reads in the mesh file in a C-binary format (4 bytes integer and 8 bytes real numbers) and a boundary condition file in ASCII format. The mesh file contains three main parts: 1) connectivity 2) vertex coordinates, and 3) boundary elements. The following code snippets describe the mesh file format:

```
=====
int nBlocks;
BinRead c_reader_; // a C-binary reader
if(!c_reader_.read(nBlocks_)) {
    cerr << "Error reading number of blocks in the binary grid file\n";
    exit(1);
}

int nelms, npts, nbelms;
if(!c_reader_.read(nelms) || !c_reader_.read(npts) ||
    !c_reader_.read(nbelms) ) {
    cerr << "Error reading nelms, npts, nbelms in the grid file\n";
    exit(1);
}

vector<vector<int>> > conn_(nelms, vector<int>()); // connectivity
vector<point_> vertex_(npts); // vertex coordinates
vector<vector<int>> > bcell_(nbelms); // boundary elements

vector<int> itmp(8); // handles less than 12 vertices per element
// connectivity
for(int i = 0; i < nelms; ++i) {
    int nvtxi;
    if( !c_reader_.read(nvtxi) ) {
        cerr << "Error reading number of vertices for element " << i << endl;
        exit(1);
    }
    if( !c_reader_.read(&itmp[0], nvtxi) ) {
        cerr << "Error reading connectivity information for element " << i <<
endl;
        exit(1);
    }
    conn_[i].resize(nvtxi);
    copy(&itmp[0], &itmp[0]+nvtxi, conn_[i].begin());
}
// vertex coordinates
vector<double> rtmp(Dim); // Dim = 2 for 2D and 3 for 3D
for(int i = 0; i < npts; ++i) {
    if( !c_reader_.read(&rtmp[0], Dim) ) {
        cerr << "Error reading grid coordinates for vertex " << i << endl;
        exit(1);
    }
}
vertex_[i] = Dim == 2 ?
    point_(rtmp[0], rtmp[1]) : point_(rtmp[0], rtmp[1], rtmp[2]);
```

```

}
// boundary elements, input vertex indices plus element number this boundary
// element
// belongs to. For example, "801 345 2" means this boundary is formed by
// vertices 801 and 345. They all belong to element number 2
if( InputParameterHolder::mixed_mesh ) {
    // read number of inputs per boundary element one by one
    int i = 0;
    while( i < nbelms ) {
        int ndata;
        if( !c_reader_.read(&ndata, 1) ) {
            cerr << "Error reading number of data in boundary element : " << i <<
endl;
            exit(1);
        }
        if( bcell_[i].size() != ndata )
            bcell_[i].resize(ndata); // mixed element and size varies
        if( !c_reader_.read(&itmp[0], ndata) ) {
            cerr << "Error reading boundary element for elem : " << i << " with "
<< ndata << endl;
            exit(1);
        }
        copy(&itmp[0], &itmp[0]+ndata, bcell_[i].begin());
        ++i;
    }
    cout << "Finished reading " << i << " boundary elements data\n";
}
else {
    // non-mixed elements
    int nvtx_bface;
    if( Dim == 2 )
        nvtx_bface = 2; // each boundary element has two vertices for 2D
    else
        nvtx_bface = gfmt_ == UNSTRUCTURED48 ? 4 : 3; // 4 vertices for hex
        and 3 for tets

    for(int i = 0; i < nbelms; ++i) {
        if( !c_reader_.read(&itmp[0], nvtx_bface + 1) ) {
            cerr << "Error reading boundary cell info for bound elem " << i <<
endl;
            exit(1);
        }
        copy(&itmp[0], &itmp[0]+nvtx_bface+1, bcell_[i].begin());
    }
}
}
=====

```

The boundary condition file has two columns below the first line indicating boundary element number (as defined in the boundary element section in the grid file described above) and its corresponding boundary segment:

```

myproblem.bc.def
1 1
2 1
3 3

```

```

4 1
5 1
6 2
7 1
8 1
9 1
10 1
11 1
12 2

```

with the first line indicating that the file `myproblem.bc.def` contains the boundary condition definition of each boundary segment.

```

segm_no  bc_flag
1        5
2        1
3        2

```

The boundary condition flag is defined according to the C++ enum defined below. The equivalent integer values start from 0 for the first entry (`REFLECT_BC = 0...`).

```

enum BCType {
    REFLECT_BC, NON_REFLECT_BC, NO_SLIP_BC_ADIAB,
    NO_SLIP_BC_CONTW, UNSTEADY_BC, FIXED_BC,
    S2D_BC, PERIODIC_BC, NO_SLIP_BC_FLUXW, ZONAL_INTERFACE_BC,
    RESERVOIR_BC, USER_DLL, NON_REFLECT_BC0,
    NON_REFLECT_BC1, NON_REFLECT_BC2, NON_REFLECT_BC3, INVALID_BC};

// in C++, enum is interchangeable with integer, i.e., REFLECT_BC = 0, NON_REFLECT_BC = 1, ...etc.
// for FIXED_BC = 5, there is one variation. A flag of 50 means, users are to specify a set of
// fixed dependent variables at this boundary.
// For example, a BC segment definition file, one can do (3D flow solver, 5 variables):
segm_no  bc_flag (e = 1/(gam*mach*mach)/(gam - 1) + rho/2(u*u + v*v), let mach = 0.2, gam = 1.4
1        50 1 0 0 0 45.1428
2        1
3        2
// for immersed_solid = true option for incompressible 2D N-S solver, one can do:
segm_no  bc_flag (p, u, v, phi)
1        50 1 0 0 1
2        1
3        2

```

For two-dimensional computations, the `mixed_mesh` option allows a mix of triangular and quadrilateral meshes. Tetrahedrons, pyramids, prisms, and hexahedrons are allowed for three-dimensional mixed meshes. To use a mixed mesh for computations, users must prepare the grid file in mixed-mesh format and set `mixed_mesh = true` (default is `mixed_mesh = false`) in the input script.

If the mesh information is available in STARCD (both 2D and 3D) format, one can use the `starcd2unicones` converter code available to prepare both grid and boundary condition files in order to run UNICONES. For MPI computations, use the `bDecompose` code to do domain decomposition, it can handle mixed mesh format.